

Stock Selection with Machine Learning

Tammy Yang
03/09/2018





- Currently working as a Credit Modeller at Investec, using both statistical and machine learning methodologies to calculate PD, LGD, EAD and other Credit Models for Investec's banking book
- Proficient in R, MatLab and SQL for model development.
- 6-year experience working as a developer and business analyst at Barclays Investment Bank
- MEng degree in Electrical and Electronic Engineering from Imperial College London
- Master in Finance degree from London Business School

The different types of learning paradigms in machine learning

Artificial Intelligence is not new!

The first AI research was founded in 1956. However it only has become well-known by the general public in the recent decade.

Machine learning is a branch of AI that uses algorithms to learn patterns in data in order to predict outcomes of future events. It has been broadly studied in its potentials in prediction of financial markets. Machine learning can be usually divided into three categories.

Unsupervised	'Learning without teaching'	Training data are not labelled. It is often used to find hidden patterns within data while the output is unknown. This method has been widely used in behavioural analysis.
Reinforcement	'Learning with semi-teaching'	Training data are not labelled. The training data is used to give a reward to the model at each time step, which the model uses to produce an optimal global strategy.
Supervised	'Learning with teaching'	Training data are labelled, allowing the algorithm to develop a model that can map input to an output. For the purpose of this project, supervised learning for binary classification is selected to develop our security selection model.

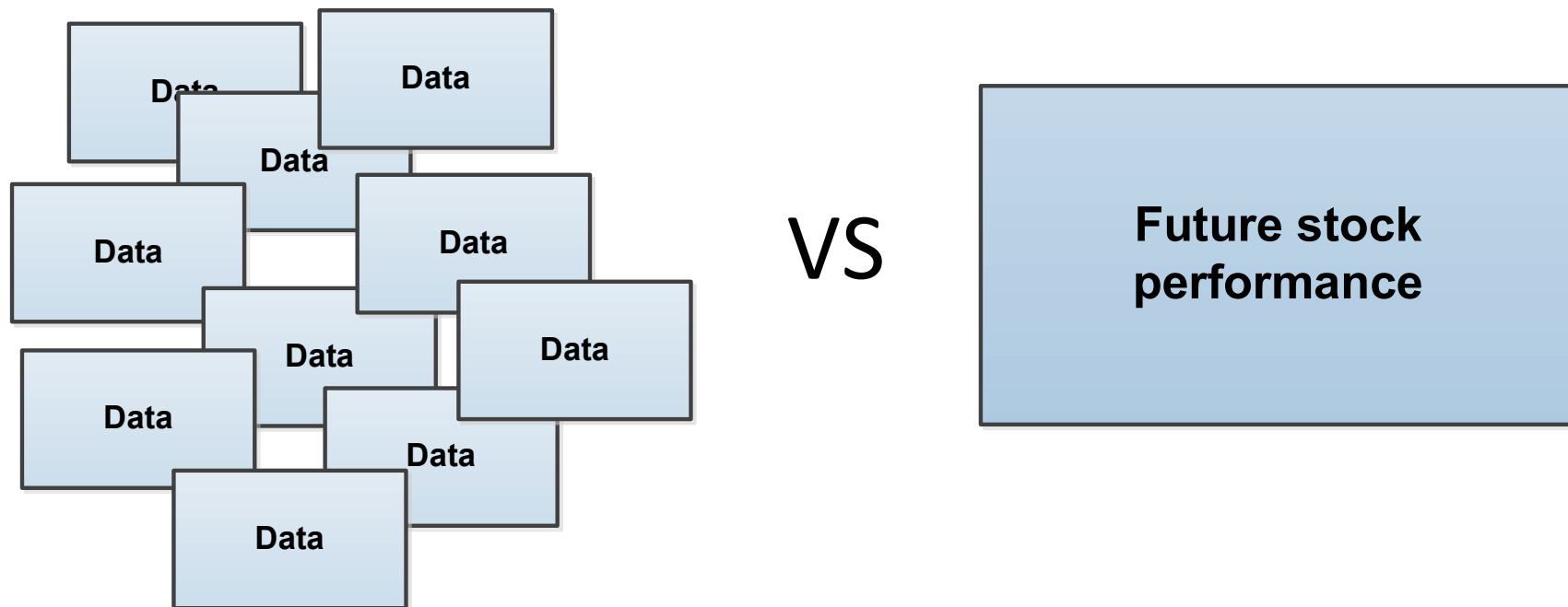
Supervised classification learning will be used for our investment strategy

- 1. Investment strategy overview**
- 2. Model development – the learning phase**
- 3. Strategy performance analysis– the prediction phase**
- 4. What's next**
- 5. Q&A**

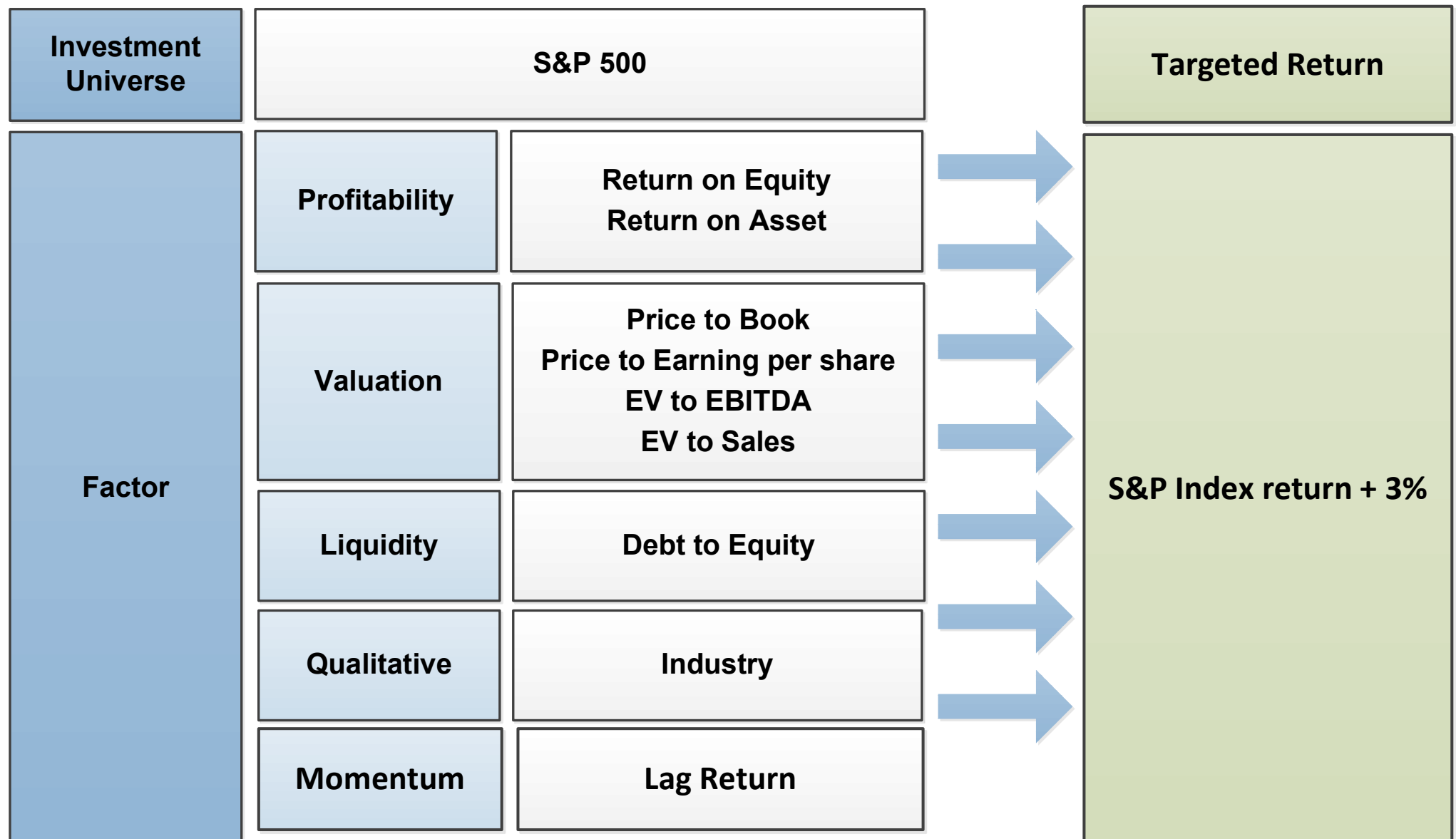
1. Strategy Overview

Our investment strategy is designed to outperform S&P 500

With machine learning, we are hoping to identify data pattern between data and stock price movement in the future.



Investment universe, factor selection, and target return



Supervised Learning

Classification

Support Vector
Machine (SVM)

Decision Tree

Why supervised classification?

Key Benefits of Supervised Classification

Ability to identify hidden pattern

Enable the computers to access the hidden pattern which has been neglected for most of the time.

Ability to process large amount of information

Machine learning model can easily consume unlimited amounts of data with timely analysis and assessment.

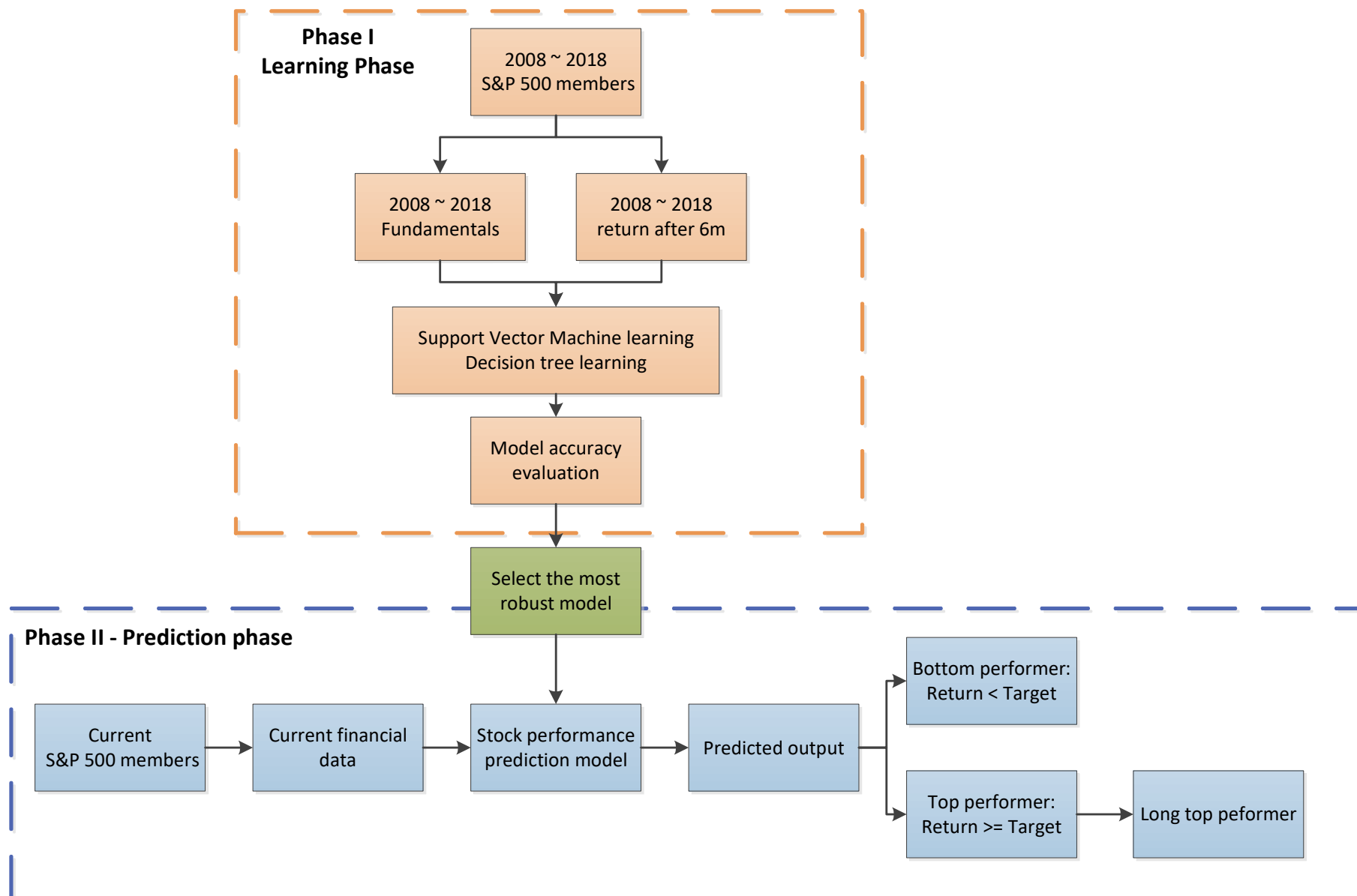
Flexible

The same algorithm/model development framework can be reused for other investment strategies with different set of model factors.

Easy to interpret

Prediction model is built based on portfolio's targeted return.

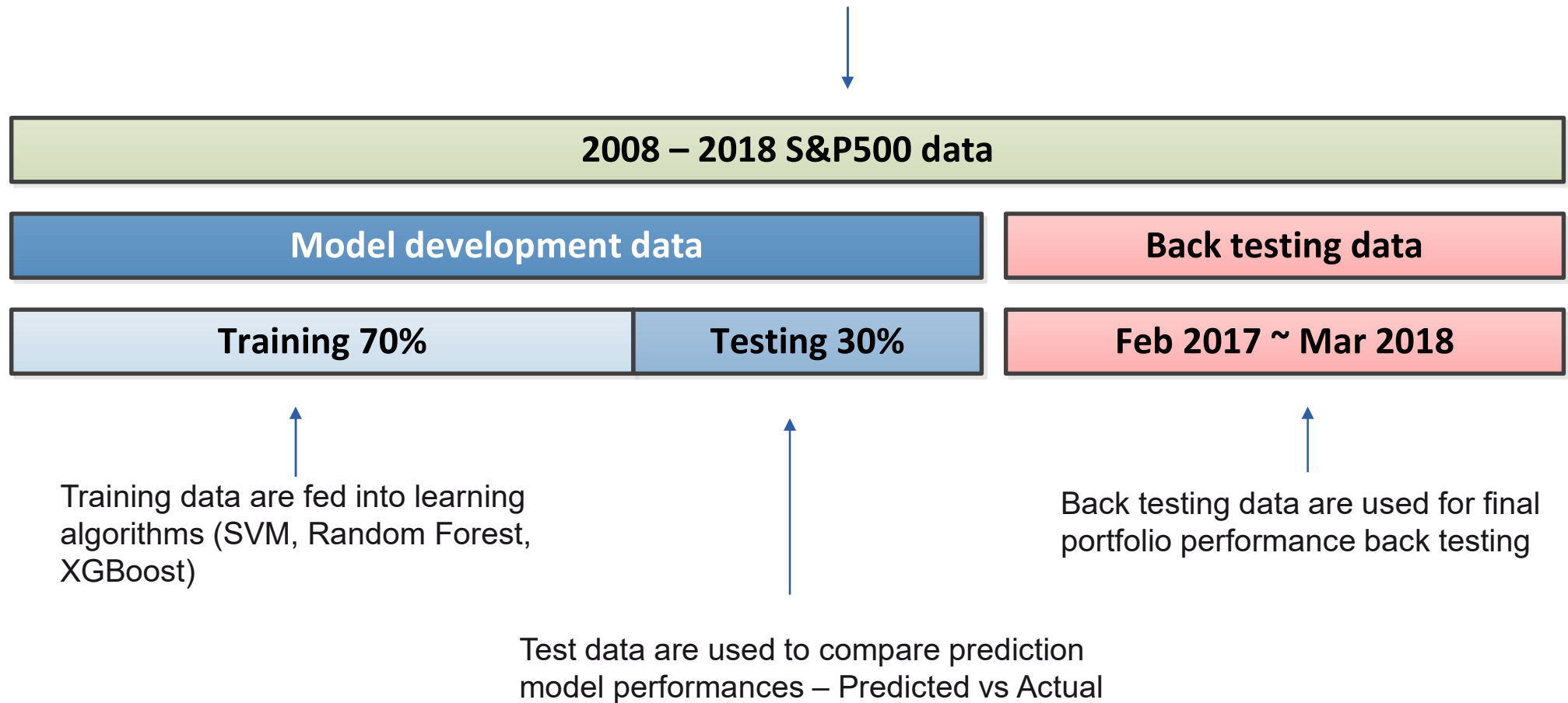
Fitting machine learning into the defined investment strategy



2. Phase I – The Learning

Data collection and transformation

Initial data collection including all defined factor and return between 2008 and 2018.



Supervised Learning

Classification

Support Vector Machine
(SVM)

Decision
Tree

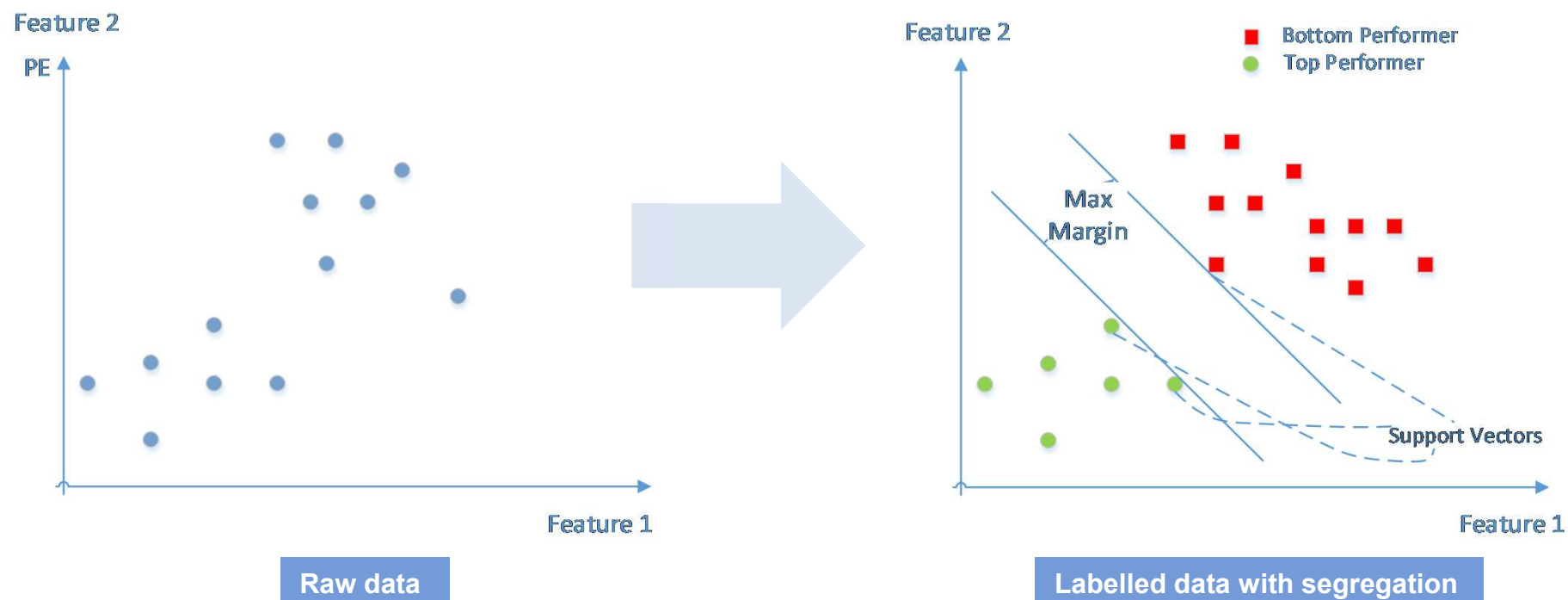
Non-linear SVM
on full dataset

Non-linear SVM
by sector

Support Vector Machine (SVM) Model

The idea of using SVM is to create a hyperplane to separate the top stock performers from the bottom stock performer, with largest margin between the closest training samples from each class. Those training sample used for separation are also known the support vectors.

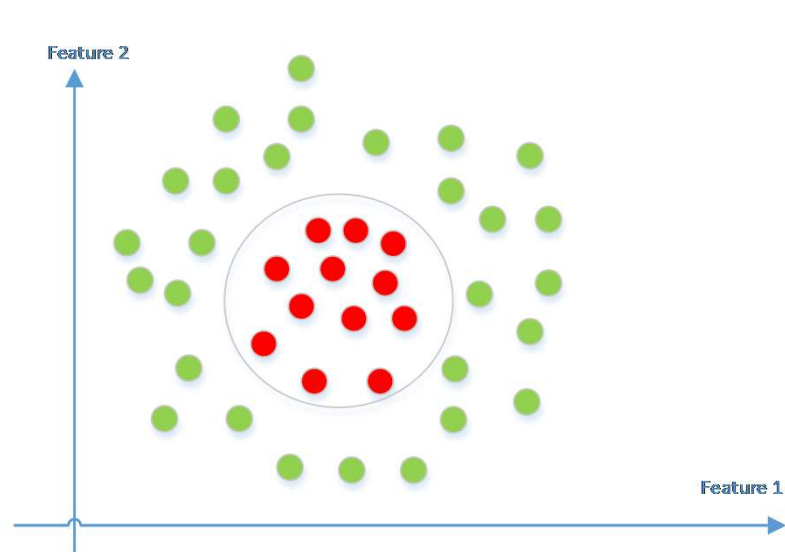
In real world, it is very difficult to separate data by linear flat hyperplane. Therefore, **non-linear SVM with Gaussian kernel and soft margin regularisation** are used in the stock performance prediction model.



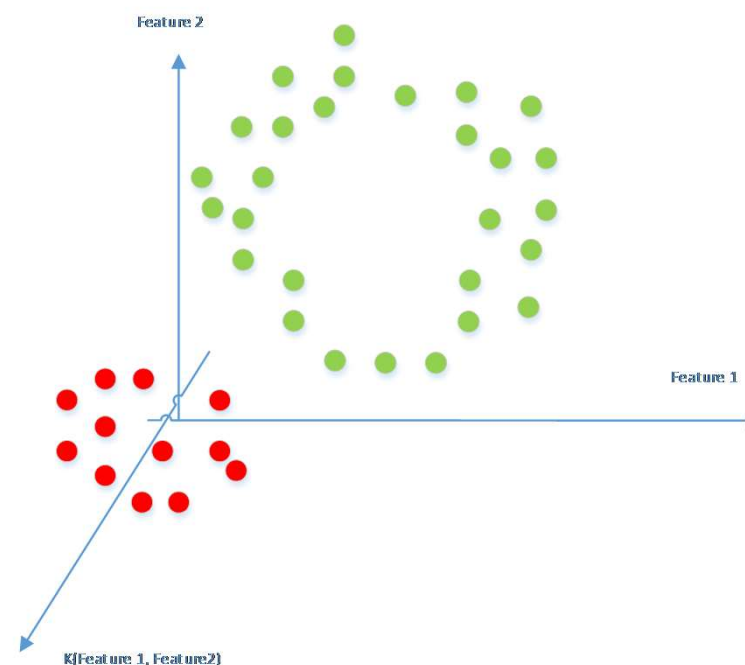
SVM Kernel parameters (k)

Kernel parameters are used to define the shape of the hyperplane. There are two different types of kernels – the polynomial kernel and the Gaussian kernel. Polynomial kernel gives a curvature separation to the data and the Gaussian kernel is normally used for non-linear data separation.

Kernel is useful for data looks like non-spreadable. This is like adding a third dimension to the 2-feature model so they can become spreadable. Kernel function is the function of inner product of Feature 1 and Feature 2.



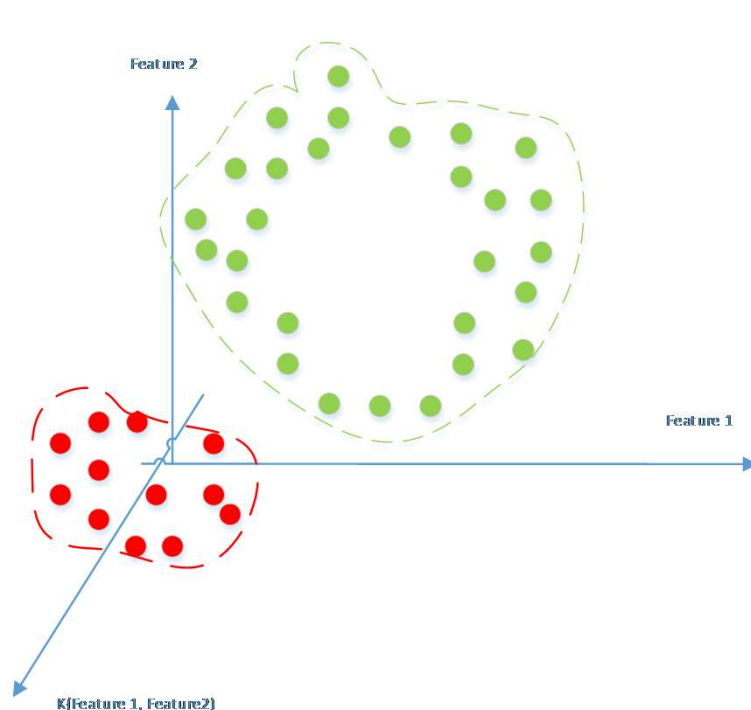
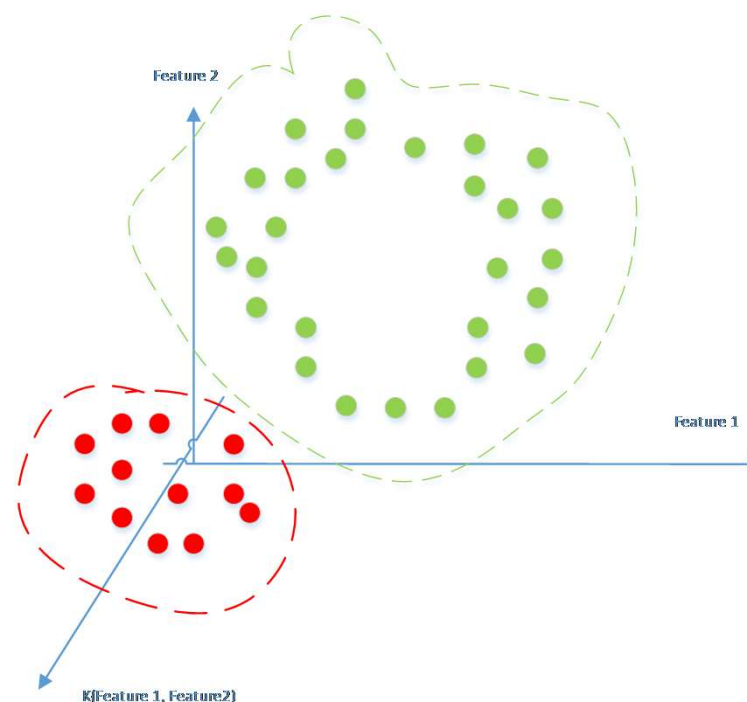
2-D view for SVM with Gaussian kernel



3-D view for SVM with Gaussian kernel

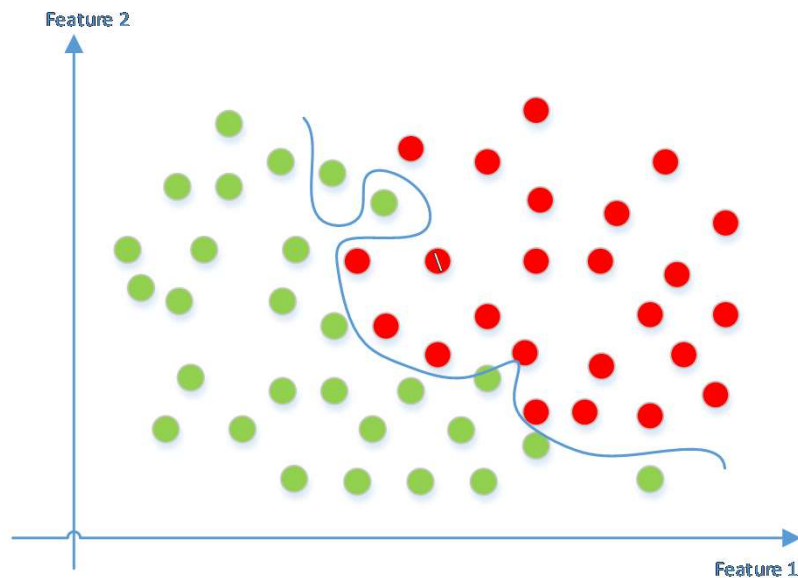
SVM γ parameter

γ parameter defines the radial influence of support vector. High γ means weak radial influence on support vector, segregation will be very close to the support vectors which means less data will be included around support vectors in the final prediction model. Low γ means strong radial influence on support factors, segregation will be further away from support vectors – more data will be included around support vectors in final prediction model.

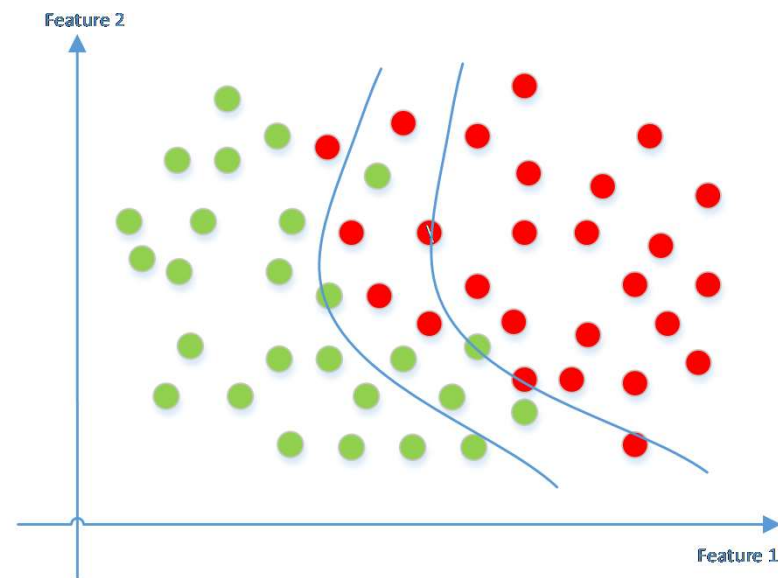
SVM with high γ SVM with low γ

SVM soft margin parameter (C)

The soft margin (C parameter) is used for generalising the separation – the smaller the C the larger the margin and thus higher proportion of non-separable samples but lower model variance.



SVM with high C – data is over fitted to training data



SVM with low C – more generalised separation to training data

Support Vector Machine – Model Hyperparameters

With 5-fold cross validation and tunelength = 10 (total number of parameter combinations that is evaluated), below is the selected hyperparameters combination

Selected hyperparameters	
Kernel Factor	0.72833
Soft margin	2
Number of support vectors	716

Supervised Learning

Classification

Support Vector
Machine (SVM)

Decision Tree

Non-linear
SVM on full
dataset

Non-linear
SVM by
sector

CART

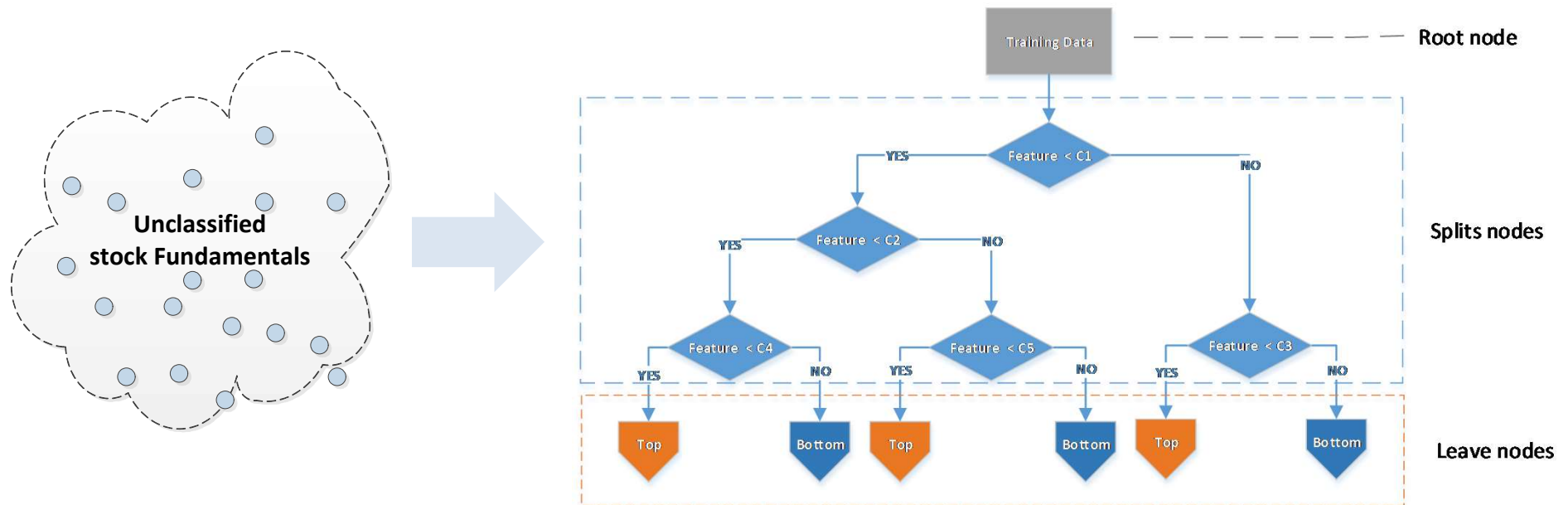
Random
Forest

Extreme
Gradient
Boosting
Tree (XGB)

Decision tree model – Single decision tree (CART)

Decision Tree Model Overview

Decision tree classification is a predictive model that uses binary rules to put input into binary baskets. For our investment strategy, the binary baskets are: i) The top performer who meets targeted return and, ii) The bottom performer who can't meet targeted return.



Decision tree model– single decision tree step-by-step

Decision Tree Model

Key steps of building a decision tree model are:

1. Grow the tree using a top-down greedy approach – making best split based on current step without considering overall optimisation
2. Split is made based on Information Ratio derived from Shannon's Entropy – which is an indicator to show how messy the data are – the higher the messier – always split at the highest information ratio.

$$H = \sum_{i=1}^N p(x_i) \log(p(x_i))$$

3. Tree pruning – a decision tree can be grown as large as possible to fit the training data, but it will also introduce overfitting issue at the same time. In order to ensure that the tree model is general enough to make prediction based on non-training data, model complexity parameter (C) is used to penalise model complexity (the total number of leave nodes).

$$\text{Loss Function} = \sum_{m=1}^T \sum_{i=1}^N p(x_{m,i}) \log(p(x_{m,i})) + C|T|$$

Easy to understand

Fast

High variance

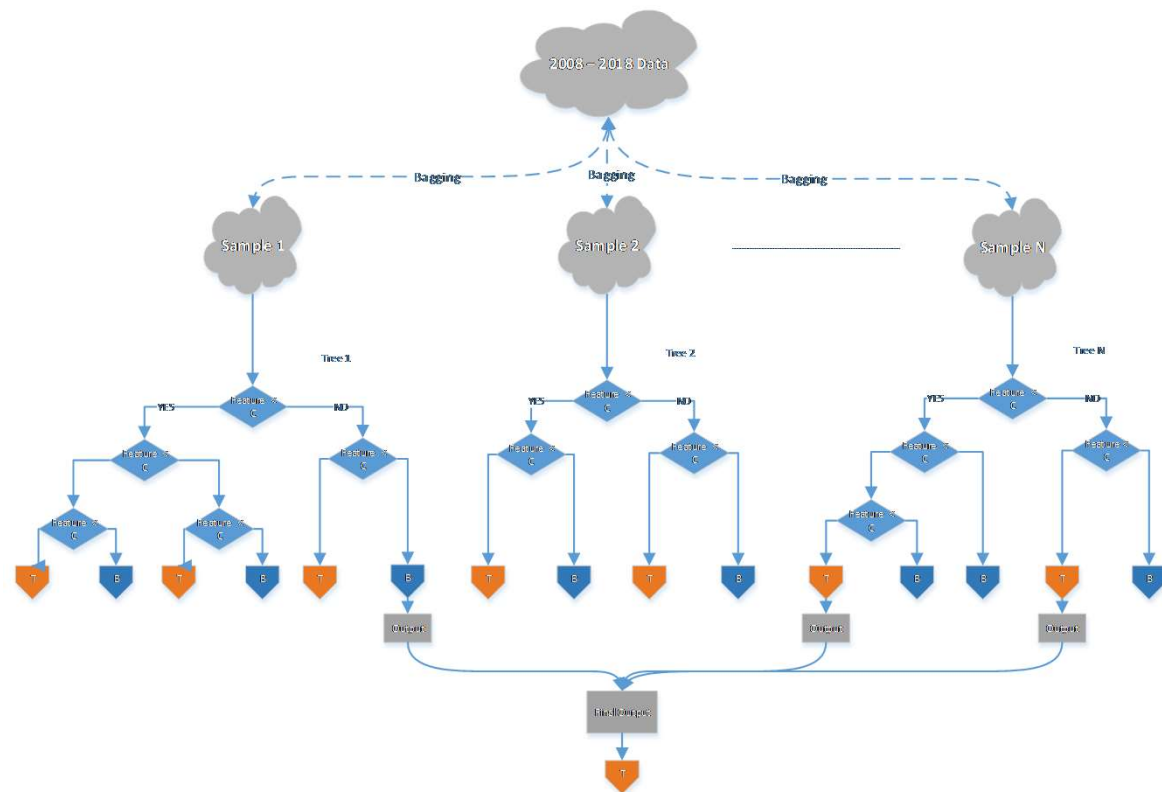
Decision Tree Model – Random Forest Learning

Random Forest Learning Model

- Ensemble learning
- Use bagging technology (resampling with replacement) to draw N samples from 2008-2018 training data.
- Create decision tree from each sample independently.
- Each tree will have a voting right to the final output at the prediction phase

Benefits

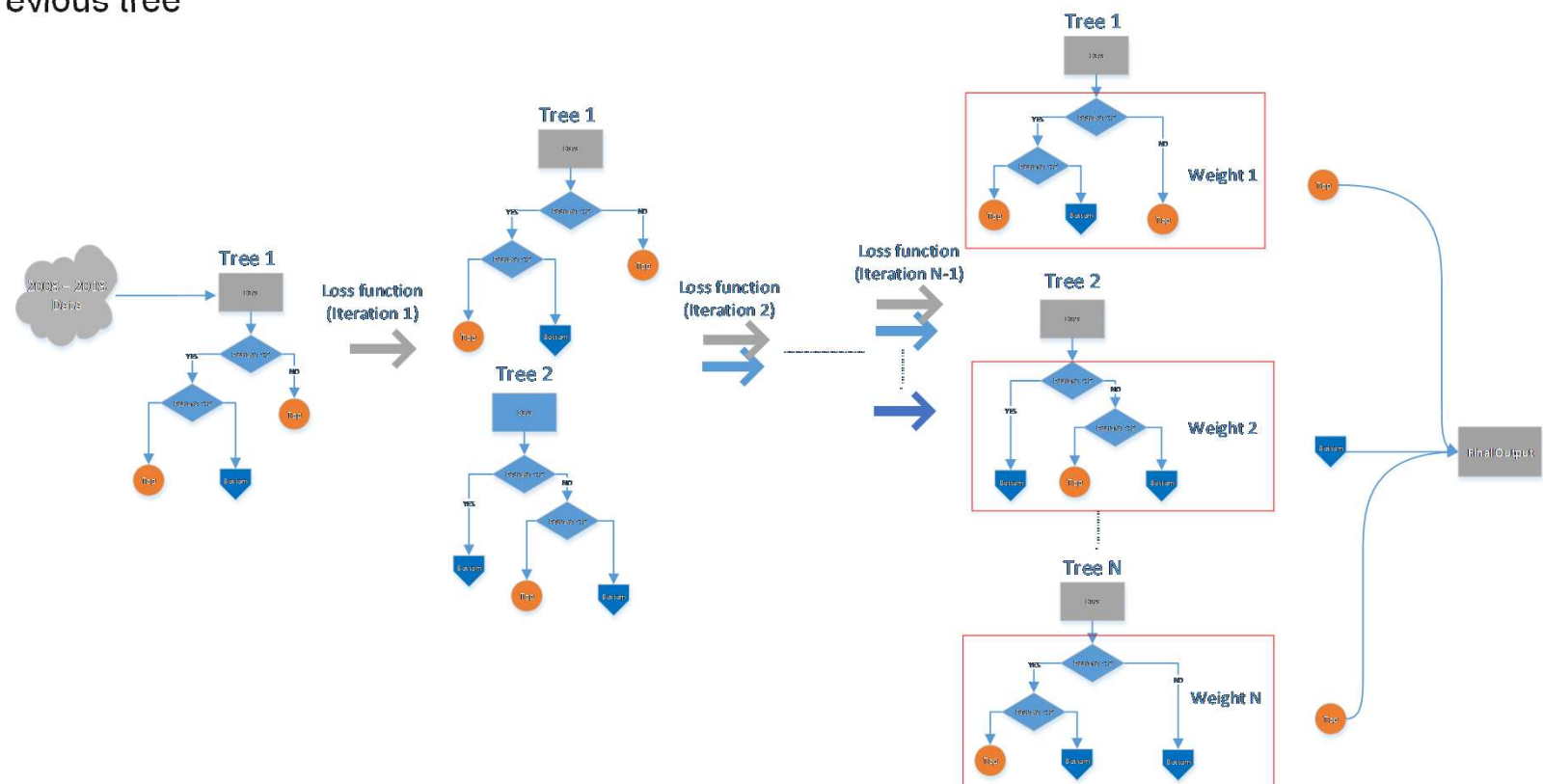
- Largely reduce high variance (over fitting) problem from single decision tree. Overall variance of the random forest will be the average of variance for each independent decision tree.



Decision Tree Model - Gradient boosting tree learning

Extreme Gradient Boosting Tree

- An ensemble of weak trees learning algorithm
- Uses boosting learning with iterative functional gradient descent optimisation
- Creates one small tree (low number of splits) at each iteration – later tree learns from the mistake of previous tree



Benefits

- Reduce model variance (over fitting) whilst handling the bias (under fitting) at the same time
- Faster

Decision Tree Model – Gradient tree boosting prediction model visualisation

Selected hyperparameters

Number of iterations	150
Max tree depth	50
Learning rate	0.06
Complexity parameters	1
Individual tree resampling %	50%
Min child weight %	10%

Hyperparameters by model type

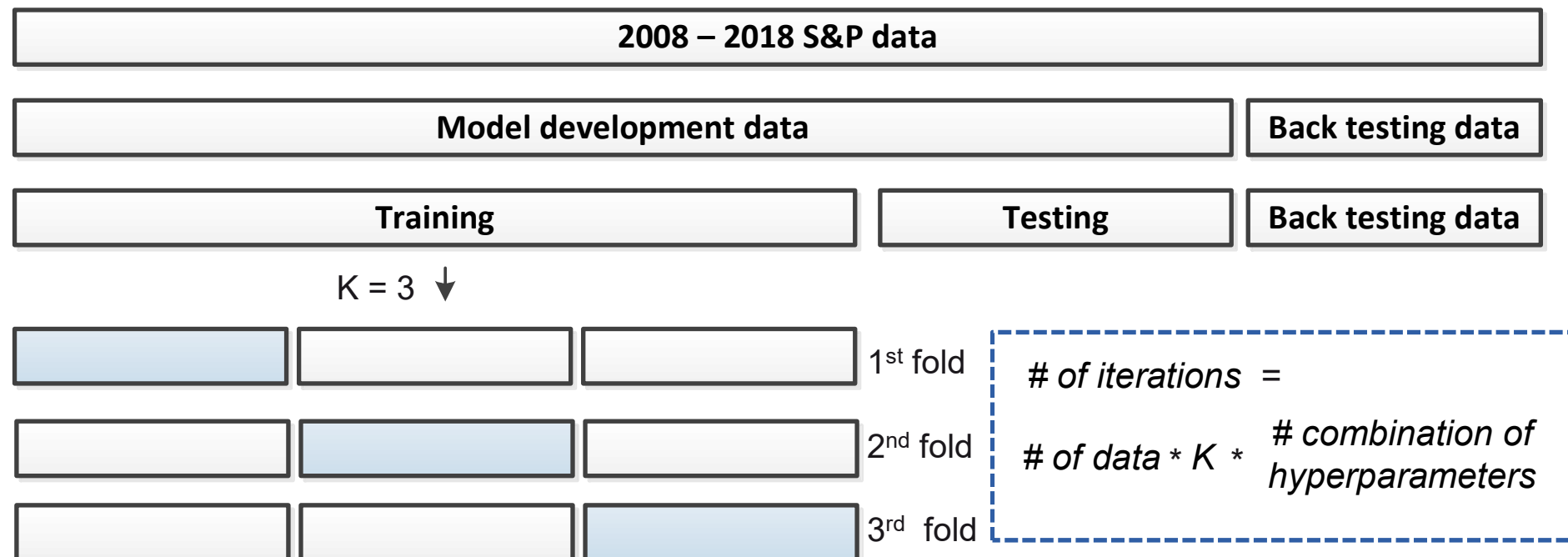
	Non-linear SVM	Random Forest	XGBoost
Model description	Classification by hyperplane	Classification by ensemble decision tree, each tree is developed independently. Final output is the average voting from the ensemble tree model	Classification by ensemble decision trees, each tree is built based on the loss function of previous tree. Final output is the weighted average of voting from the ensemble tree mode
Hyperparameters	Kernel (Sigma)	Number of trees	Number of iterations
	Soft margin (C)	Maximum tree depth	Maximum tree depth
		Sample size from population	Sample size from population
			Learning rate for gradient descend

K-fold cross validation on training data is used to identify the optimal hyperparameters combination

Identifying the optimal hyperparameter combination

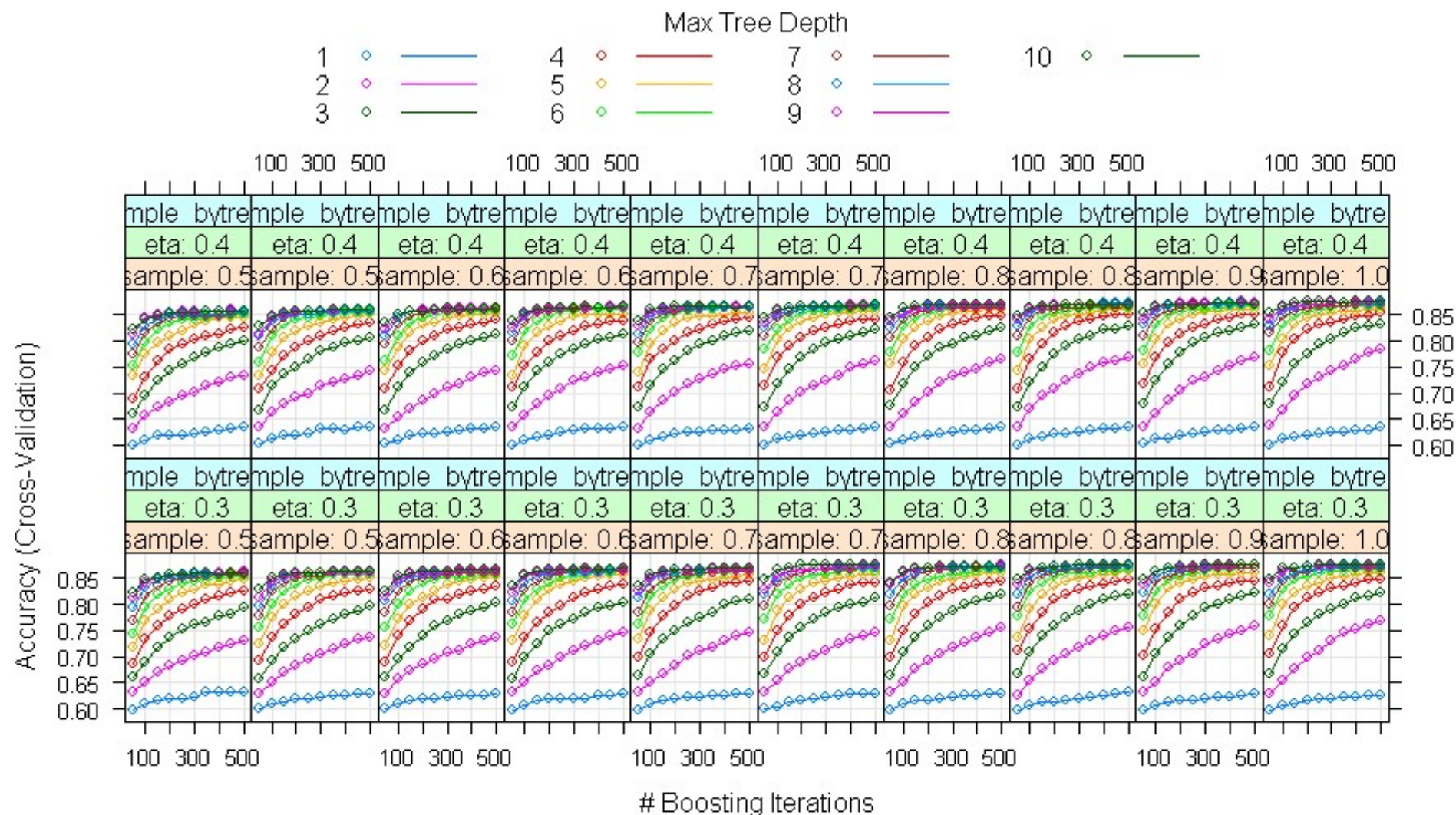
K-fold cross validation to optimise model

Original training data will be randomly partitioned into k number of subsamples. Of the k sub samples, 1 subsample will be retained as the validate data for testing and the remaining k-1 subsample are used for training with hyperparameter combination. Each hyperparameter combination will go through all 3 fold validation and the final validation result will be the average of all three. Large K with big data set can be very computational expensive. Sometime model optimisation will be converged over increasing K, so K is not necessary the larger the better.



Identifying the optimal hyperparameter combination

XGBoost 10-fold cross validation example



Confusion Matrix

	Actual Top	Actual Bottom
Predicted Top	True Top	False Top
Predicted Bottom	False Bottom	True Bottom

$$\text{True Top Rate} = \frac{\text{True Top}}{\text{True Top} + \text{False Bottom}}$$

$$\text{False Top Rate} = \frac{\text{False Top}}{\text{False Top} + \text{True Bottom}}$$

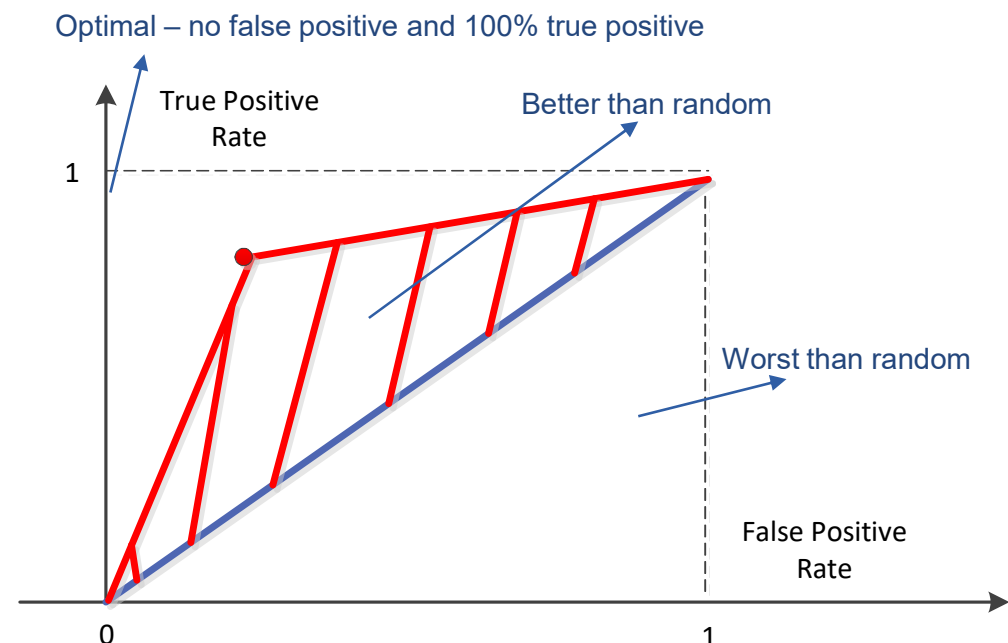
Receiver operating characteristic

Simple accuracy rate can be sometime misleading especially for imbalance data with high negative rate (in our case, high number of bottom performers). The ROC curve illustrates the diagnostic ability of binary classifier, plotting by TPR against FPR:

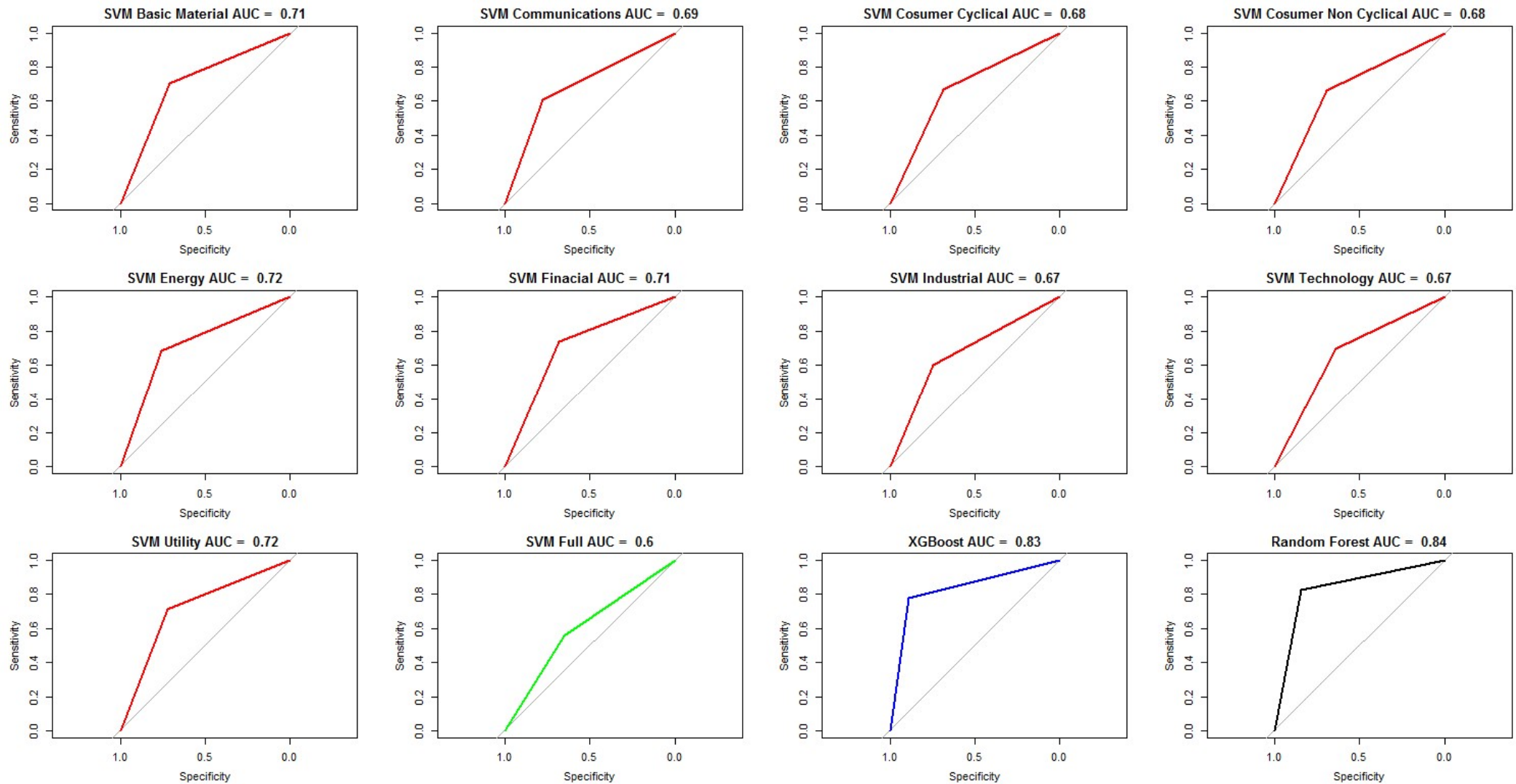
- TPR – how many top performers have been correctly predicted
- FPR – how many bottom performers have been misclassified as top performers

AUC, the area under curve, is used to measure model performance:

- 1 – Best model performance
- 0 – The same as random select
- Less than 0 – worst than random select

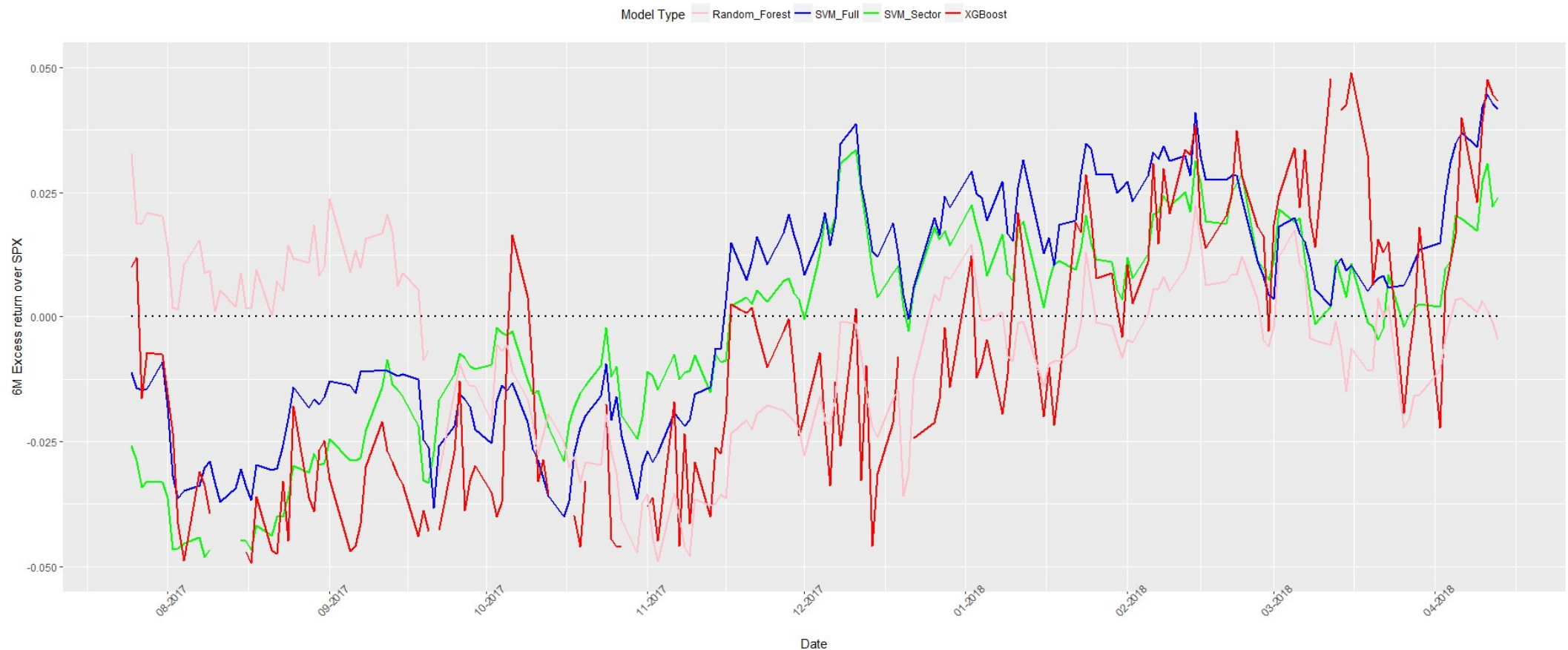


AUC are calculated and used to select the best performing prediction model



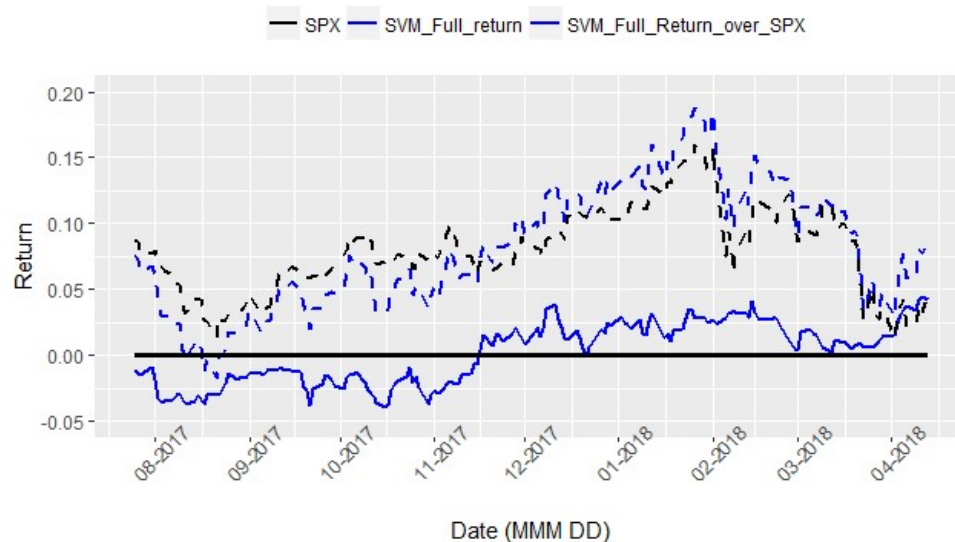
3. Phase II – The Prediction

Portfolio Return over SPX - Back testing 2017

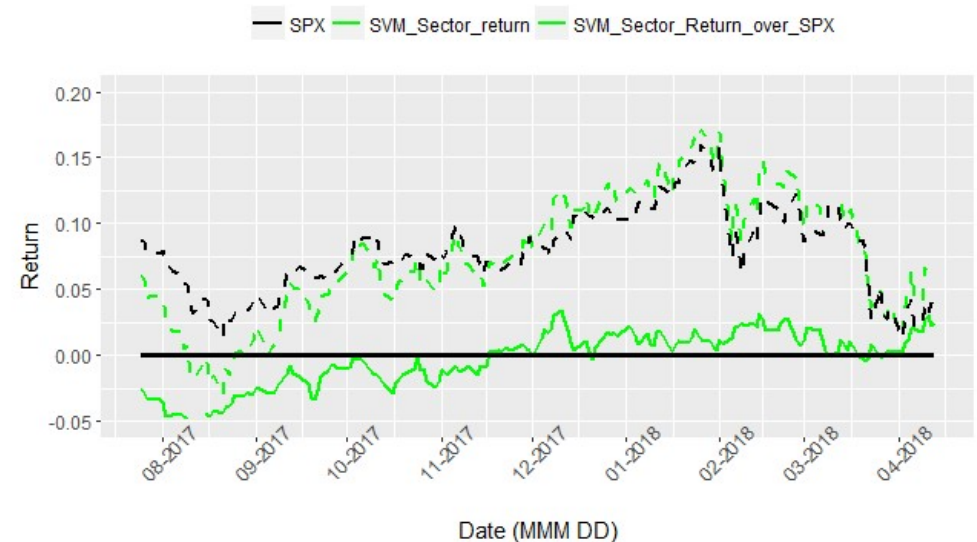


Back Testing – Return analysis by model type

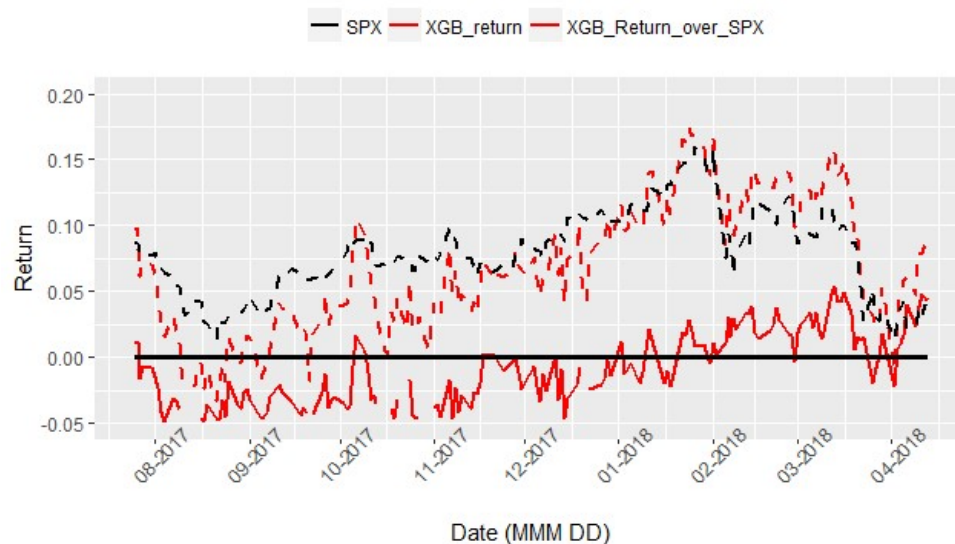
Portfolio Return Analysis (SVM full) - Back testing 2017



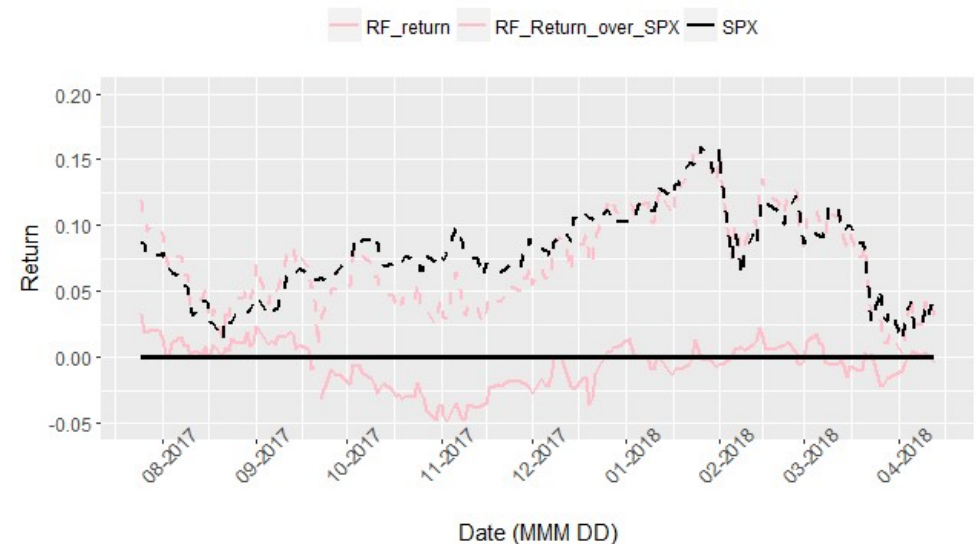
Portfolio Return Analysis (SVM Sector) - Back testing 2017



Portfolio Return Analysis (XGB) - Back testing 2017

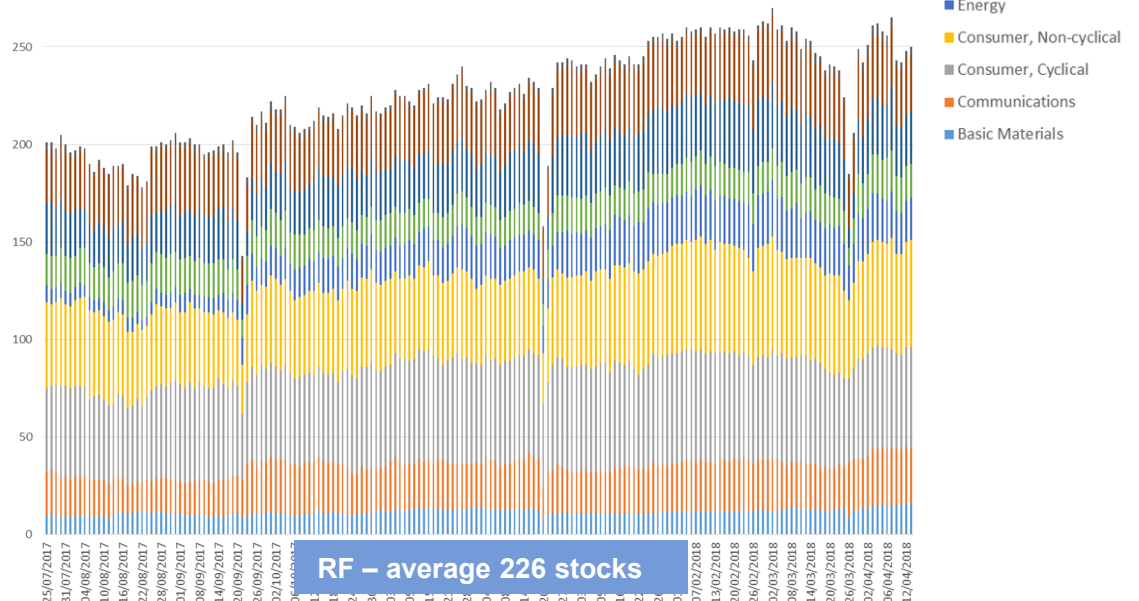
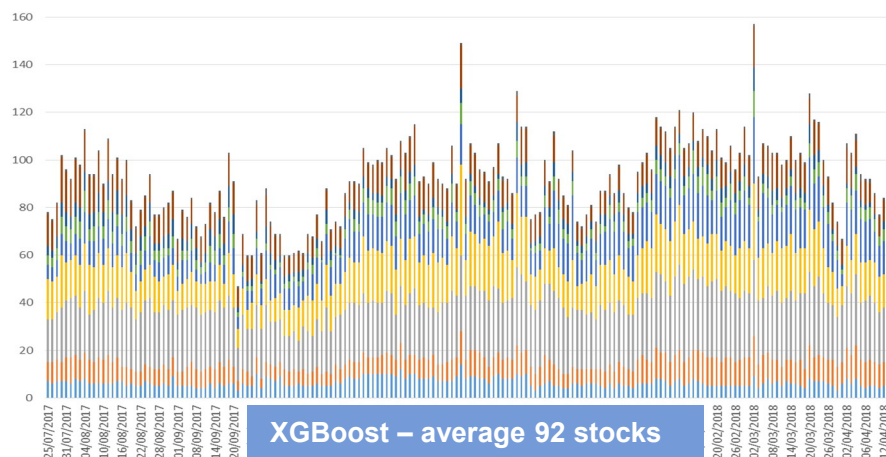
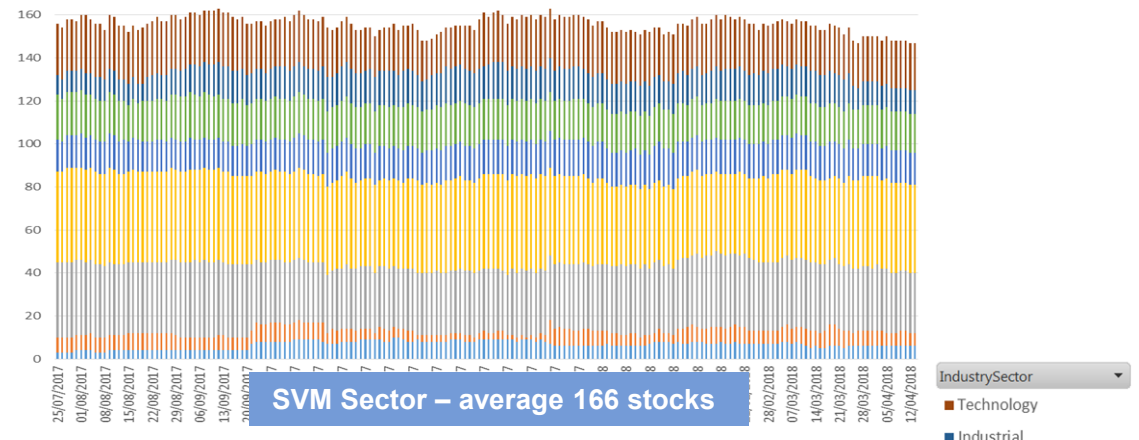
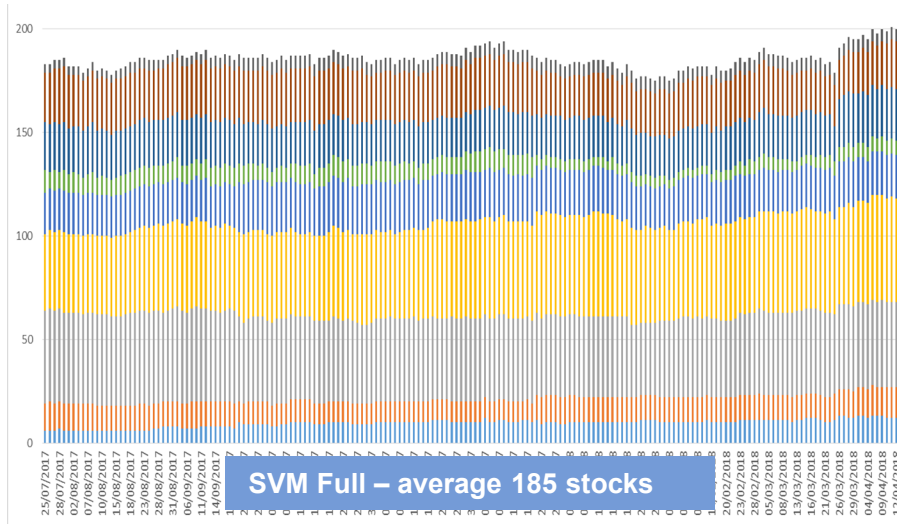


Portfolio Return Analysis (Random Forest) - Back testing 2017



Phase II – The Prediction

Portfolio construction



Portfolio performance overview by model type

SVM Full	
Mean Return	16.084%
Return Standard Deviation	6.600%
Risk Free Return	3.004%
Sharpe Ratio	1.9819034
Beta	0.6067301

SVM Sector	
Mean Return	15.080%
Return Standard Deviation	6.558%
Risk Free Return	3.004%
Sharpe Ratio	1.8413272
Beta	0.6277203

XG Boost	
Mean Return	13.395%
Return Standard Deviation	7.062%
Risk Free Return	3.004%
Sharpe Ratio	1.4713876
Beta	0.5272457

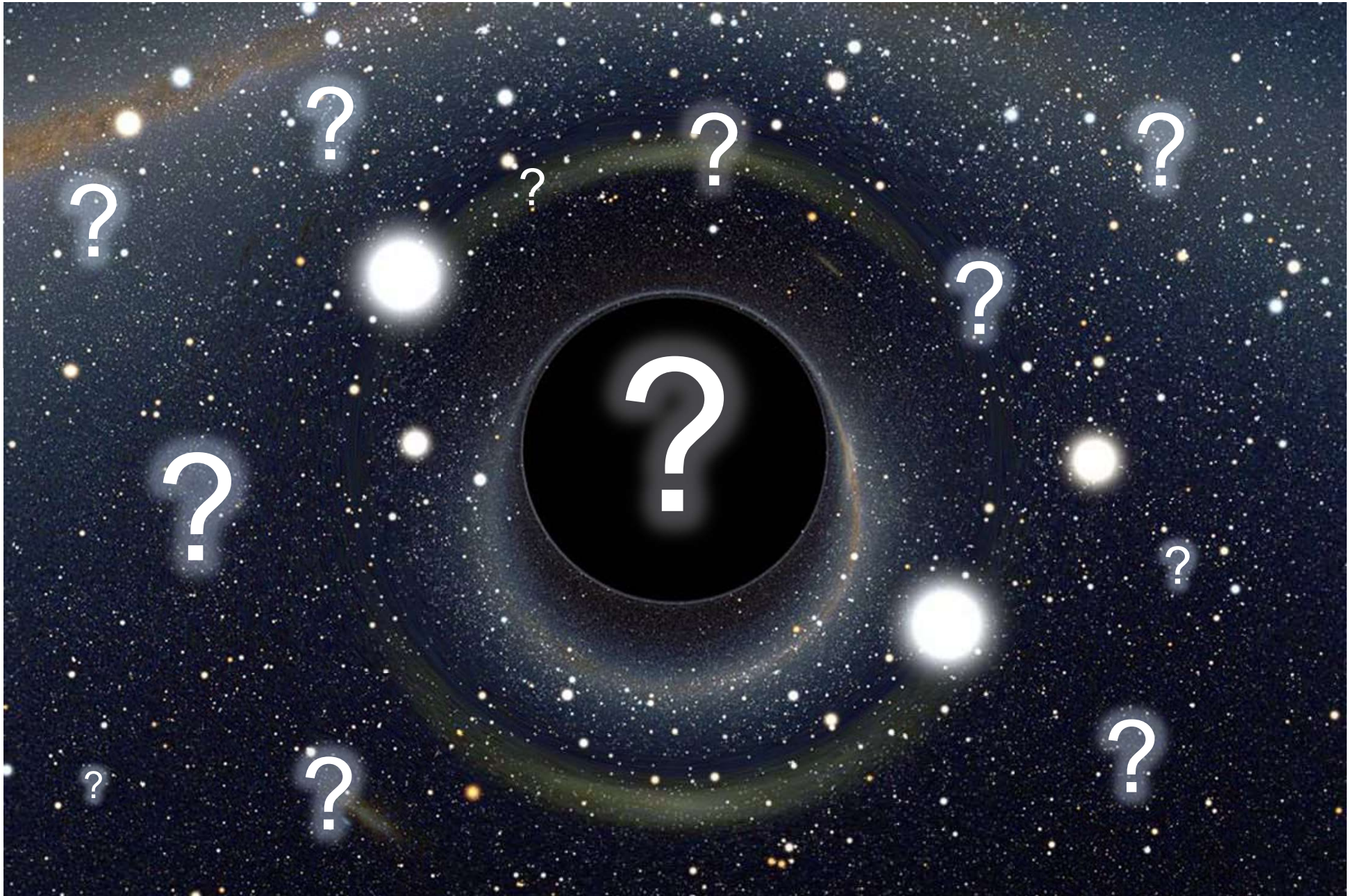
Random Forest	
Mean Return	14.469%
Return Standard Deviation	5.006%
Risk Free Return	3.004%
Sharpe Ratio	2.29019
Beta	0.7804397

Selected as the prediction model!

Machine learning is not a black box, but it is a black hole.....

This is not the end, there are a lot more we can do to improve the model:

- Proper initial data analysis
- Combine supervised and unsupervised learning
- Combine XGBoost and SVM
- More features - we defined more than features at the beginning
- More data – expand the universe to 1990-2018 to include more than 1 economy downturn
- Change the investment universe – may be emerging market
- Transaction cost
- Holding period
- Portfolio weight
- More model tuning



5. Appendix

R libraries for data analytics and modelling

Usage	Library	Usage	Library
Imbalanced data handling	DMwR ROSE	XGB modelling	xgboost
Parallel running	parallel doParallel Foreach	Tree modelling	rpart ISLR
Model building	caret	Analytics	PerformanceAnalytics np
SVM modelling	e1071	ROC curve	plotROC pROC

Risk metrics are calculated on an ex-ante basis and reported regularly, which include but are not limited to volatility, beta exposures, maximum drawdowns, Value at Risk measures, scenario analysis and stress testing.

Expected Volatility Target: 10 – 15%	Measured by the standard deviation of the difference between daily portfolio return and mean return of the portfolio $\sigma_p = \sqrt{\frac{\sum_{i=1}^n (R_{P,i} - \mu)^2}{N}}$
Value-at-Risk Target 95% 1Year VaR: 10%	A 95% confidence level of VaR has been used to quantify the level of financial risk within our portfolio
Maximum Drawdown Target: 10%	Maximum peak to trough loss of a portfolio

Portfolio return assessment

Mean annualized return

- We calculate this by scaling up the mean return as reported in a non-compound way.

Sharpe ratio

- We define this as the mean annualized return divided by the standard deviation of annualized returns.

Performance presentation

- Both equally weighted and value weighted are recorded

Sharpe Ratio	Calculated as the portfolio excess return over standard deviation of the portfolio. It measures how much excess return can the portfolio generates over per extra unit of volatility $\text{Sharpe ratio} = \frac{R_{p,i} - R_{f,i}}{\sigma_p}$
Alpha	Measures the excess return over a risk-adjusted basis. CAPM model has been used to calculate alpha: $\alpha = R_{p,i} - R_{f,i} - \beta * (R_{m,i} - R_{f,i})$
Beta	Systematic risk of the portfolio that related to benchmark $\beta = \frac{\text{Covariance}(R_p, R_m)}{\text{Variance}(R_M)}$

